

How to Run the Automation Script (Part 2 - Selenium, Java, TestNG)

1. Project Setup (Maven + Eclipse)

This project is built using:

- Java
- Selenium WebDriver
- TestNG
- Maven (dependency management)
- Eclipse IDE

2. Prerequisites

Ensure the following are installed before execution:

- Java JDK 8 or higher
- Eclipse IDE
- Google Chrome (latest version)
- ChromeDriver (compatible with Chrome version)
- Maven configured in Eclipse
- TestNG plugin installed in Eclipse

3. Maven Dependencies

Make sure your pom.xml includes:

- Selenium Java
- TestNG

(If already added, no extra configuration is required.)

4. How to Run the Tests

Since this project does **NOT use TestNG XML**, tests are executed directly from the Test class.

Option 1: Run from Eclipse (Recommended)

1. Open Eclipse IDE
2. Navigate to:
3. SaforaContactUsTest.java
4. Right-click inside the class file
5. Select:
6. Run As → TestNG Test
7. Browser will launch automatically and execute all test cases in priority order

Option 2: Run Individual Test Cases

1. Open the class file
2. Locate any method annotated with:
3. @Test
4. Right-click on the specific test method
5. Select:
6. Run As → TestNG Test
7. Only that specific test case will execute

5. Execution Flow

When the script runs:

1. Chrome browser is launched
2. Navigates to: <https://safora.se/en/>
3. Clicks on **Contact Page**
4. Executes test scenarios:
 - Positive form submission
 - Empty form validation
 - Invalid email test
 - Missing field validations
 - Boundary tests (name length, phone length)
 - CAPTCHA handling scenarios

6. Expected Output

During execution, you will observe:

- Console logs for each step (field entry, clicks, validations)
- Browser automation flow on Safora Contact Form
- CAPTCHA handling (manual or automated attempt)
- Success or validation error messages
- TestNG results in Eclipse console (Pass/Fail status)

***Important Notes

- CAPTCHA may require **manual intervention** depending on Google verification
- Additional delays were added in Selenium scripts using Thread.sleep() to handle reCAPTCHA loading and multiple verification challenges, ensuring stable execution and preventing timing related test failures due to asynchronous behavior.
- Some negative test cases may behave differently based on backend validation
- Ensure stable internet connection for page loading and CAPTCHA rendering
- Execution order is controlled using:
 - priority = 1,2,3...

***Important Note on Automation Scripts

In this project, I have attached two separate automation scripts to demonstrate both basic and framework based test automation approaches using Selenium.

1. Simple Selenium Java Script (Non TestNG)

This script is designed to execute a basic happy path scenario for the Contact Us form.

Purpose:

- To demonstrate basic Selenium WebDriver automation using Java
- To perform a straightforward **positive test case (valid form submission)**
- To verify that the Contact Us form works correctly with valid input data
- To show simple execution flow without using any test framework

2. TestNG Framework Script

This script is implemented using the **TestNG framework** to provide a structured and scalable automation approach.

Purpose:

- To demonstrate professional test automation using TestNG
- To organize test cases using annotations (@Test, @BeforeMethod, etc.)
- To include both positive and negative test scenarios
- To validate form behavior under different input conditions

Framework Behavior Summary

- No XML file is required for execution
- Test execution is controlled directly by TestNG annotations
- Each test runs independently using @BeforeMethod and @AfterMethod
- Browser is launched fresh for each test case